

User Documentation

User Guide

Designing, Simulating, and Flying Model Rockets

How to build rocket designs, manage motors, and run flight simulations with `qtrocket-cli` — from a first launch to scripted design studies.

Covers	Part I: the command-line interface (<code>qtrocket-cli</code>). Part II (the Qt6 GUI) is planned.
Version	QtRocket v0.2 (<code>development</code> branch, July 2026)
Units	SI throughout: meters, kilograms, seconds, Newtons; altitude is z , ground at $z = 0$
Audience	Rocketeers and simulation users; no C++ knowledge required

About this guide

QtRocket is an open-source model-rocket flight simulator: a C++ simulation core shared by a Qt6 desktop application, a command-line REPL called `qtrocket-cli`, and a standalone 3D design viewer called `qtrocket-visualizer`. This guide teaches the command-line interface — today the most feature-complete way to drive QtRocket, and the only way to *author* rocket designs. Everything you build here saves to `.qrd` files that the GUI and the visualizer open directly.

The guide is organized as a short concepts primer (Sections 1 and 2), five hands-on tutorials (Sections 3 to 7), and a complete command reference (Section 8). The tutorials are the heart of it: each one builds something real, flies it, and reads the results, introducing commands as they earn their place.

Every transcript in this guide is real

The session transcripts in this guide are not typed-up approximations: each one is the captured output of the released `qtrocket-cli` binary running the tutorial scripts included alongside this document (Appendix A). The screenshots are captures of `qtrocket-visualizer` displaying the `.qrd` files those same sessions saved, and the flight plots are drawn from the CSV files they wrote. Where a long payload is shortened, the elision is marked in *[bracketed gray italics]* — nothing else is edited. If you run the scripts, you will get these numbers.

Reading the transcripts. Lines you type are shown in **bold blue**. The CLI answers every command with a line starting **OK** (success) or **ERR** (failure); those and the occasional **WARN** are colored the way you should read them. Muted `#` gray lines are comments and banners the CLI prints or ignores.

Conventions. All quantities are SI — meters, kilograms, seconds, Newtons; densities are kg/m^3 . Altitude is the z axis with the ground at $z = 0$. Motor designations follow the standard hobby impulse classes (A, B, C, ...), where each letter doubles the total impulse of the one before it.

Contents

About this guide	ii
I The Command-Line Interface	1
1 Getting started	1
1.1 Building qtrocket-cli	1
1.2 Four ways to run it	1
1.3 The output protocol and exit codes	1
1.4 A ninety-second first session	2
2 Concepts: the design tree, placement, and what persists	2
2.1 A rocket is a tree of parts	2
2.2 Placement: seat, stations, gap	3
2.3 Motors and the motor database	3
2.4 What persists, and where	4
3 Tutorial 1 — First Flight: Three Fins and a Nose Cone	5
4 Tutorial 2 — The Motor Lab: Two Databases, One Airframe, Four Motors	8
5 Tutorial 3 — Placement: Seats, Stations, and Reading the Red	12
6 Tutorial 4 — The Physics Knobs: Atmospheres, Integrators, Timestep, Angle	16
7 Tutorial 5 — One-Button Flights: Scripts, Exit Codes, and CSV	20
8 Command reference	23
8.1 Session and meta commands	23
8.2 Motor database commands	24
8.3 Flight configuration commands	25
8.4 Environment and numerics commands	27
8.5 Design commands	28
8.6 Run and inspection commands	31
II The Graphical Interface (Planned)	33
A Reproducing this guide	34

Part I

The Command-Line Interface

1 Getting started

1.1 Building `qtrocket-cli`

QtRocket builds with CMake presets; Qt6 is the only system dependency (and the CLI itself does not use Qt — Qt is needed to build the sibling GUI targets). Everything else is fetched and pinned by CMake, so the first configure is slow once:

```
cmake --preset release
cmake --build --preset release
```

The binary lands at `build/cli/qtrocket-cli`. The examples in this guide assume it is on your `PATH` or invoked by that path.

1.2 Four ways to run it

1. **Interactively** — run `qtrocket-cli` with no arguments and type commands at it. It reads lines from standard input until end-of-file or a `quit`.
2. **From a pipe** — `printf 'help\n' | qtrocket-cli`. Same reader, no terminal required.
3. **From a script file** — `qtrocket-cli flight.cli` runs the commands in the file. This is how the tutorial scripts are run.
4. **One-liner** — `qtrocket-cli -c "listmotors"` runs a single command string in a fresh process. `-help` and `-version` do what you expect.

State does not survive between processes

Each invocation is a fresh session. `qtrocket-cli -c "setmotor B6"` followed by `qtrocket-cli -c "launch"` does *not* fly a B6 — the second process never saw the first command. Put multi-step work in one script file, one pipe, or one interactive session; use `.qrd` design files ([Section 7](#)) to carry work *between* sessions.

1.3 The output protocol and exit codes

The CLI is built to be read by both humans and scripts. Every command answers with a line beginning `OK <cmd>` on success or `ERR` on failure; data payloads (motor listings, part trees, state samples) usually follow the `OK` header unprefix. Two exceptions keep parsers honest: `help` prints its #-prefixed listing *before* its terminating `OK help`, and comment or blank lines produce no output at all. Two more prefixes appear only around flights: `CSV <path>` names the trajectory file `launch` wrote, and

WARN: `run aborted (...)` flags a flight that ended for a safety reason rather than a normal landing. Comment lines start with `#`; the log system is pinned to errors-only with an `[ERROR]` prefix so log noise can never masquerade as command output.

The process exit code summarizes the whole session: **0** if every command succeeded, **1** if any command reported `ERR`, and **2** for a bad invocation (unreadable script file, malformed `-c`). A failing command never kills the session — the REPL reports it and keeps reading — but the exit code remembers. [Section 7](#) turns this into one-line regression guards.

1.4 A ninety-second first session

```
# QtRocket CLI - type 'help' for commands, 'quit' to exit.
help
# commands:
#   loadmotors <file.rse|file.eng>   import a motor database
#   savedb <file.qmd>               save the motor database to a file
[40 more help lines --- the full list is Section 8]
OK help
launch
ERR launch: no motor set (use loadmotors + setmotor first)
quit
OK bye
```

That `ERR` line is the CLI teaching you its own rules: a flight needs a motor, and motors come from a database you load. That is exactly where Tutorial 1 begins.

If you get lost

`help` prints every command with a one-line summary, and `status` ([Section 8](#)) shows the current session configuration — motor, masses, atmosphere, integrator, timestep — at any moment.

2 Concepts: the design tree, placement, and what persists

2.1 A rocket is a tree of parts

A QtRocket design is a tree: one root part (usually the nose cone) with children attached to it, children attached to those, and so on. `newdesign` creates the root; `addpart` attaches a child to any existing part, addressed by the `name=` you gave it, by its numeric id, or by the word `root`. Composite mass, center of gravity, and inertia are computed from the tree — there is no separate “mass budget” to maintain.

Address parts by name in scripts

Ids are assigned in session order and *reset when a design is reloaded*, so a saved script that says `removepart 5` may bite a different part tomorrow. Names are stable: give every part a `name=` and use it.

Four part types are constructible today. All lengths are meters, densities kg/m^3 — never millimeters.

Parameter values are parsed strictly: `length=0.25abc` is rejected, not truncated. Unknown keys are skipped so newer files keep working on older builds — but a token without `=` is an error.

Type	Required keys	Optional keys
NoseCone	baseRadius length density	solid (default true), wallThickness, name
BodyTube	outerRadius length density	innerRadius (default 0 = solid rod), name
FinSet	finCount rootChord tipChord span thickness bodyRadius density	sweep (default 0), name
HollowSphere	outerRadius density	innerRadius (default 0 = solid), name

Table 1. Constructible part types and their `key=value` parameters. A motor is *not* a factory part — it attaches through `setmotor`.

2.2 Placement: seat, stations, gap

Every `addpart` link records *physical intent*, not baked coordinates: a **seat** kind plus optional station and gap overrides. Stations are fractions of a part’s live length — `0` is the aft plane, `1` the fore plane — so editing a part’s length later moves every seam with it.

Seat	Meaning	gap means
Abut	Child’s fore plane against the parent’s aft plane — how a body tube hangs under a nose cone. The default seat.	signed forward standoff (0 = touching)
NestInBore	Child slides into the parent’s bore from the fore rim — couplers, payloads.	insertion depth (≥ 0 ; = child length $\Rightarrow$ flush)
OnSurface	Child sits on the parent’s outer wall — fins. Align the axial position with <code>parentStation/childStation</code> .	signed forward standoff along +z, like Abut

Table 2. The three seat kinds. The seat sets a canonical station pair first; explicit `parentStation/childStation/gap` keys then override individually.

`checkdesign` is the arbiter of whether the assembled geometry makes physical sense: it reports *overlaps* (a part intruding into another’s solid material) with the offending part ids, the radial penetration, and the *z* location, and *air gaps* (a stack that is not contiguous) with the uncovered *z* interval and its size. Tutorial 3 breaks a design twice on purpose to show both messages — and shows the visualizer painting the offender red.

Where the deep theory lives

The placement model — stations, seats, the resolver, and its diagnostics — has its own design whitepaper in `docs/PartPlacementDesignLatex/`. This guide teaches the working vocabulary; the whitepaper holds the proofs.

2.3 Motors and the motor database

Motors live in a session database, filled from three sources that merge: local RockSim files (`loadmotors file.rse`), local RASP files (`loadmotors file.eng`), and QtRocket’s own `.qmd` format (`loaddb/savedb`). An online search (`tcsearch`, needs network) adds results from thrustcurve.org into the same database. `setmotor` selects a motor by its exact common name (B6, G80T); `listmotors` shows what is loaded.

2.4 What persists, and where

- **The design** — part tree, placement links, drag coefficient, reference area, and the selected motor *by name* — is written by `savedesign` to a versioned `.qrd` XML file and restored by `loaddesign`. If the named motor is not in the database at load time, the design still loads without one, and the **OK** line says so: `OK loaddesign: file.qrd (no motor)`.
- **The motor database** persists only via `savedb` to `.qmd`.
- **Session configuration** — atmosphere, gravity, integrator, timestep, launch velocity and angle — is deliberately *not* part of the design file. Scripts restate it; that keeps a `.qrd` a description of a rocket, not of one particular flight.

3 Tutorial 1 — First Flight: Three Fins and a Nose Cone

You will build the classic first rocket — a nose cone, a body tube, and three fins — verify it, fly it on a B6, and read every line of the flight report.

Setup

Work in an empty directory containing a copy of the small Estes-class motor database that ships with QtRocket:

```
mkdir flight1 && cd flight1
cp <qtrocket>/tests/data/motors/estes_small.qmd .
qtrocket-cli
```

The complete session below is also included as `scripts/tutorial1.cli`, runnable with `qtrocket-cli tutorial1.cli`.

Load motors, find one

```
# QtRocket CLI - type 'help' for commands, 'quit' to exit.
loaddb estes_small.qmd
OK loaddb: 13 new motors from estes_small.qmd (13 total)
listmotors B
OK listmotors: 2 shown (filter="B")
B4  avg=4.17N  Itot=5Ns
B6  avg=5.03N  Itot=4.33Ns
```

Two commands, two habits. `loaddb` fills the session motor database from a `.qmd` file and answers with how many motors arrived — every command answers **OK** or **ERR**, and that is the entire protocol. `listmotors` takes an optional substring filter (case-sensitive); `B` narrows the thirteen bundled motors to the B-class. The two numbers per row — average thrust and total impulse — are the two numbers that matter when choosing a motor.

Build the airframe

```
newdesign NoseCone name=nose baseRadius=0.0098 length=0.10 density=700
OK newdesign: root NoseCone id=2
addpart nose BodyTube name=body innerRadius=0.0091 outerRadius=0.0098 length=0.25 density=700
OK addpart: BodyTube id=3 under 2
addpart body FinSet name=fins finCount=3 rootChord=0.04 tipChord=0.02 span=0.03 sweep=0.02 thickness=0.003
↪ bodyRadius=0.0098 density=700 seat=OnSurface parentStation=0 childStation=0
OK addpart: FinSet id=4 under 3
checkdesign
OK checkdesign: 3 parts, contiguous, no overlaps
listparts
OK listparts:
[2] NoseCone "nose"  m=0.0070401 kg
[3] BodyTube "body"  m=0.00727357 kg
[4] FinSet "fins"    m=0.00567 kg
-- composite: mass=0.0199837 kg, cg_z=-0.200687 m (t=0)
```

Read the `addpart` lines closely; they carry the whole design grammar:

- `newdesign` starts the tree with a root part — here a solid conical nose, 0.0098 m base radius, 0.10 m long, at 700 kg/m³ (a plausible balsa-and-paper blend).

- The body tube names its parent: `addpart nose BodyTube ....` Parents are addressed by the `name=` you gave them — names, not ids, are the scriptable handle. No placement keys are needed: the default seat (**A**but) hangs the tube's fore rim flush under the nose's aft plane.
- The fins use `seat=OnSurface` with `parentStation=0 childStation=0`: sit on the body's outer wall, aft edges aligned — fins flush with the bottom of the tube.

`checkdesign` then renders the verdict that matters: 3 parts, contiguous, no overlaps. `listparts` shows the tree with each part's computed mass and the composite mass and center of gravity — derived from geometry and density, not typed in.

Fly it

```
setmotor B6
OK setmotor: B6
setdrag 0.75
OK setdrag: 0.75
setarea 0.000302
OK setarea: 0.000302 m^2
status
OK status:
  motor      = B6
  dry_mass   = 1 kg
  drag_coeff  = 0.75
  ref_area   = 0.000302 m^2
  velocity   = 0 m/s
  angle      = 0 deg (from vertical)
  atmosphere = Constant Atmosphere
  gravity     = Constant Gravity
  integrator  = Runge-Kutta 4th Order
  database    = 13 motors
launch
OK launch: 1457 steps, t_final=14.560s
  apogee     = 241.405 m @ t=6.330s
  max_speed  = 97.562 m/s @ t=0.820s
  downrange  = 0.000 m
  landing    = t=14.560s, pos=(0.000, 0.000, -0.314)
CSV /tmp/flight1/qtrocket_run.csv
```

`setmotor` attaches the B6 by its common name. `setdrag` and `setarea` set the drag coefficient and aerodynamic reference area — 0.75 and the body tube's frontal disc ($\pi r^2 \approx 3.02 \times 10^{-4} \text{ m}^2$) are honest starting values for a small sport model. `status` plays back the full session configuration before committing to flight. One row deserves a squint: `dry_mass = 1 kg` is only the mirror of the last `setmass` — an initial placeholder here, never applied, since this session hasn't called `setmass` at all. Your rocket flies on the part tree's computed $\approx 20 \text{ g}$ composite plus the motor, as `listparts` showed. (On a design, `setmass` immediately overwrites the *root part's* own mass — rarely what you want on a multi-part rocket; trust the tree.)

The `launch` report is the payoff, one line per question you would ask at the field: how high (apogee), how fast (`max_speed`, reached just after burnout), how far downwind (`downrange` — zero, this was a straight-up flight), and where and when it came down. The `CSV` line names the full trajectory file — every timestep, every state — written for plotting or analysis (Section 7 uses it).

Keep it

```
savedesign first_rocket.qrd
OK savedesign: first_rocket.qrd
quit
OK bye
```

`savedesign` writes the rocket — tree, placement, drag config, and the motor by name — to a `.qrd` file. Open it in the visualizer to see what you built (Fig. 1):

```
qtrocket-visualizer first_rocket.qrd
```

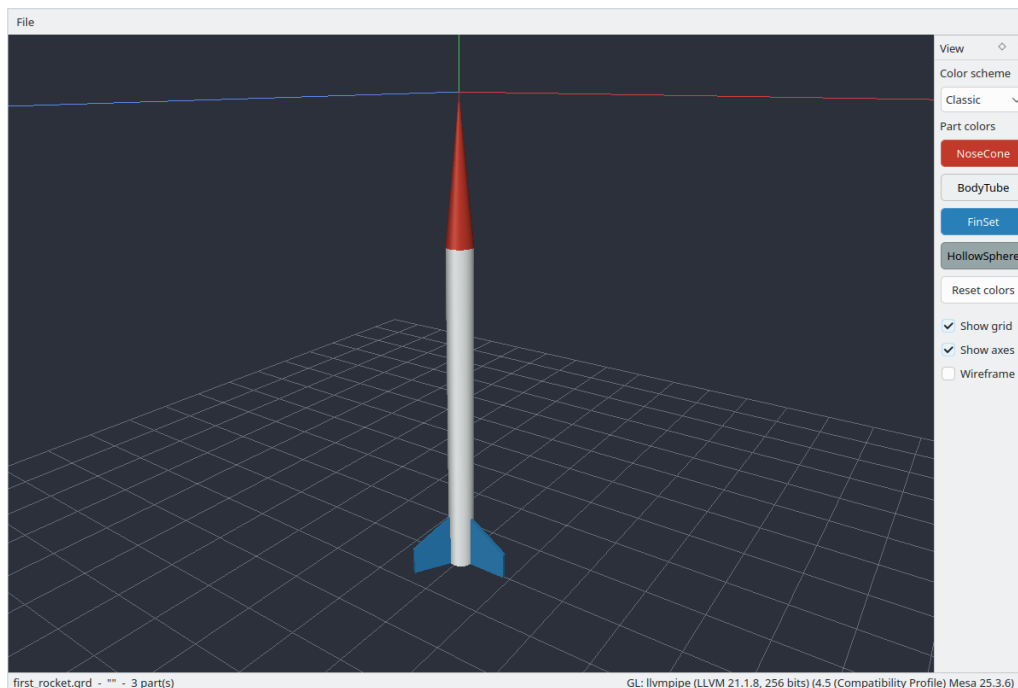


Figure 1. Your first rocket in `qtrocket-visualizer`: conical nose, 18 mm-class body, three swept fins flush with the aft rim. Orbit with the left mouse button, zoom with the wheel.

What you learned

`loaddb` → `listmotors` → `newdesign` → `addpart` (names as handles, seats as intent) → `checkdesign` → `setmotor` → `launch` → `savedesign`. That eight-step spine is every QtRocket session; the rest of this guide is variations.

4 Tutorial 2 — The Motor Lab: Two Databases, One Airframe, Four Motors

One airframe, four motors, four very different flights. This tutorial is about motor management: the two local file formats, how databases merge, and the cleanest experiment QtRocket offers — swap the motor, change nothing else, and watch total impulse turn into altitude.

Setup

An empty directory with both bundled motor files:

```
cp <qtrocket>/tests/data/motors/estes_small.qmd .
cp <qtrocket>/data/Aerotech.rse .
qtrocket-cli tutorial2.cli      # or type along interactively
```

Two files, one database

```
# QtRocket CLI - type 'help' for commands, 'quit' to exit.
loaddb estes_small.qmd
OK loaddb: 13 new motors from estes_small.qmd (13 total)
loadmotors Aerotech.rse
OK loadmotors: 249 motors from Aerotech.rse
listmotors D1
OK listmotors: 4 shown (filter="D1")
D11  avg=9.4N  Itot=17.49Ns
D12  avg=10.21N Itot=16.84Ns
D13W avg=11.284N Itot=19.183Ns
D15T avg=14.012N Itot=19.617Ns
listmotors E30
OK listmotors: 1 shown (filter="E30")
E30T avg=32.387N Itot=39.512Ns
```

`loaddb` reads QtRocket's native `.qmd`; `loadmotors` imports vendor formats — RockSim `.rse` here, RASP `.eng` works the same way. Both `merge` into one session database: the 13 Estes-class motors and the 249 AeroTech motors are now one searchable pool, which is why the `D1` filter catches entries from both files at once. Filters are plain case-sensitive substring matches — `E30` pins down one motor family.

A mid-size airframe

```
newdesign NoseCone name=nose baseRadius=0.013 length=0.09 density=900
OK newdesign: root NoseCone id=2
addpart nose BodyTube name=body innerRadius=0.0121 outerRadius=0.013 length=0.30 density=1000
OK addpart: BodyTube id=3 under 2
addpart body FinSet name=fins finCount=3 rootChord=0.05 tipChord=0.025 span=0.03 sweep=0.02 thickness=0.003
↪ bodyRadius=0.013 density=900 seat=OnSurface parentStation=0 childStation=0
OK addpart: FinSet id=4 under 3
checkdesign
OK checkdesign: 3 parts, contiguous, no overlaps
listparts
OK listparts:
[2] NoseCone "nose"  m=0.0143351 kg
[3] BodyTube "body"  m=0.0212906 kg
```

```
[4] FinSet "fins" m=0.0091125 kg
-- composite: mass=0.0447382 kg, cg_z=-0.209509 m (t=0)
```

Nothing new grammatically — a 26 mm-diameter, 30 cm tube under a stubby nose — but note the composite mass in `listparts`: about 45 g of airframe. Keep that number in mind as the motors get heavier.

The ladder: C11, D12, E30T, F32T

```
setdrag 0.75
OK setdrag: 0.75
setarea 0.000531
OK setarea: 0.000531 m^2
setmotor C11
OK setmotor: C11
launch
OK launch: 1501 steps, t_final=15.000s
  apogee    = 257.089 m @ t=6.600s
  max_speed = 100.609 m/s @ t=0.770s
  downrange = 0.000 m
  landing   = t=15.000s, pos=(0.000, 0.000, -0.419)
CSV /tmp/flight1/qtrocket_run.csv
save flight_C11.csv
OK save: /tmp/flight1/flight_C11.csv
setmotor D12
OK setmotor: D12
launch
OK launch: 2037 steps, t_final=20.360s
  apogee    = 447.674 m @ t=8.140s
  max_speed = 145.186 m/s @ t=1.590s
  downrange = 0.000 m
  landing   = t=20.360s, pos=(0.000, 0.000, -0.350)
CSV /tmp/flight1/qtrocket_run.csv
save flight_D12.csv
OK save: /tmp/flight1/flight_D12.csv
setmotor E30T
OK setmotor: E30T
launch
OK launch: 2603 steps, t_final=26.020s
  apogee    = 721.934 m @ t=8.540s
  max_speed = 308.837 m/s @ t=0.890s
  downrange = 0.000 m
  landing   = t=26.020s, pos=(0.000, 0.000, -0.175)
CSV /tmp/flight1/qtrocket_run.csv
save flight_E30T.csv
OK save: /tmp/flight1/flight_E30T.csv
setmotor F32T
OK setmotor: F32T
launch
OK launch: 3007 steps, t_final=30.060s
  apogee    = 940.544 m @ t=9.770s
  max_speed = 326.873 m/s @ t=1.300s
  downrange = 0.000 m
  landing   = t=30.060s, pos=(0.000, 0.000, -0.255)
CSV /tmp/flight1/qtrocket_run.csv
save flight_F32T.csv
OK save: /tmp/flight1/flight_F32T.csv
```

Each rung is the same three commands: `setmotor`, `launch`, `save` (which copies the trajectory CSV before the next flight overwrites it). The geometry never changes; only the motor does. Four flights, four apogees:

Motor	Class	Apogee	Time to apogee
C11	C	257.1 m	6.6 s
D12	D	447.7 m	8.1 s
E30T	E	721.9 m	8.5 s
F32T	F	940.5 m	9.8 s

Each impulse class doubles the total impulse of the one before it — and the apogees march accordingly, sub-doubling because drag grows with speed. [Figure 2](#) overlays the four saved trajectories.

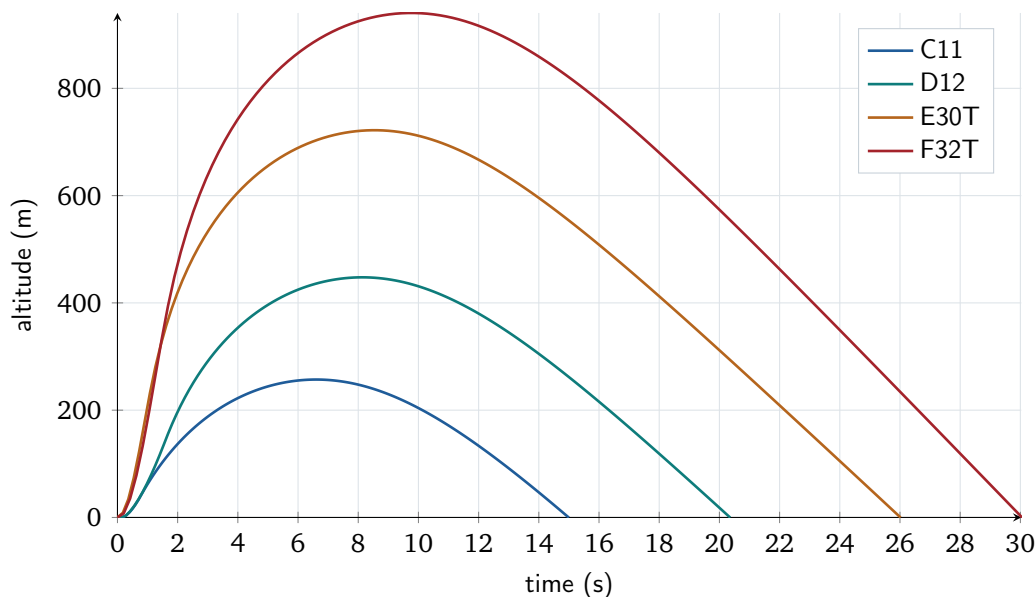


Figure 2. The motor ladder: one airframe flown on four motors, plotted from the four `saved` trajectory files. Impulse class doubles rung to rung; drag keeps the apogees from doubling with it.

Keep the collection

```
savedb my_flight_box.qmd
OK savedb: 262 motors to my_flight_box.qmd
savedesign motor_lab.qrd
OK savedesign: motor_lab.qrd
quit
OK bye
```

`savedb` writes the whole merged database — both files' motors plus anything an online search added — to one `.qmd`, so the next session starts with a single `loaddb`. The saved design ([Fig. 3](#)) records the *last* selected motor by name.

What you learned

`.qmd` vs `.rse/.eng`; databases merge rather than replace; `listmotors` filters are substrings;

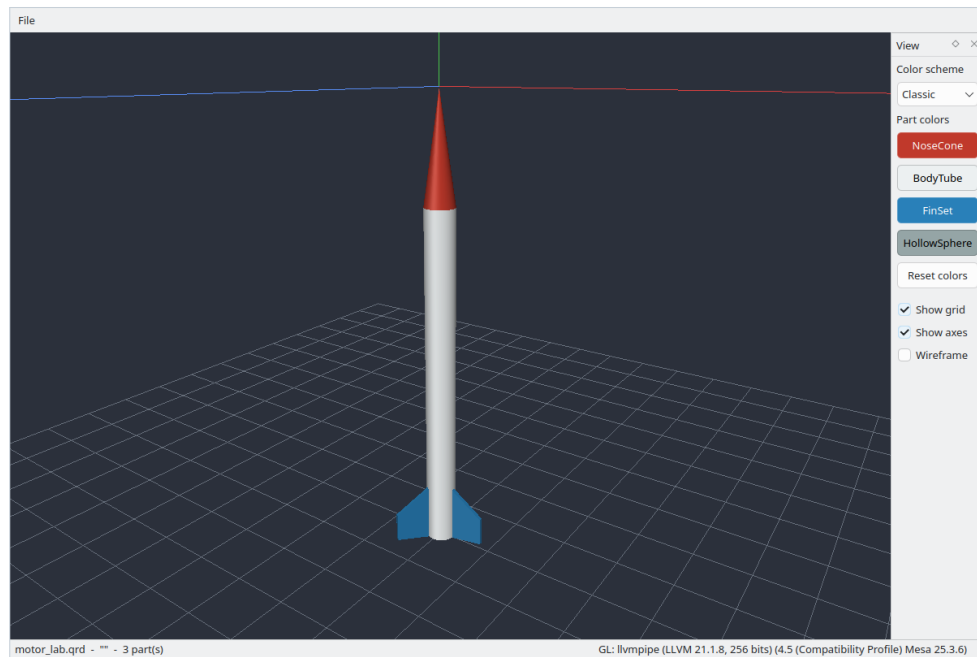


Figure 3. `motor_lab.qrd`: the 26 mm test airframe. Same silhouette all day — only the motor changed.

`setmotor` swaps propulsion without touching geometry; `save` preserves a flight's CSV before the next `launch` overwrites it; `savedb` persists the merged pool.

5 Tutorial 3 — Placement: Seats, Stations, and Reading the Red

This is the placement tutorial: all three seat kinds, internal parts, and — deliberately — two broken designs, because `checkdesign`'s error messages and the visualizer's red paint teach the model faster than any prose. By the end you will have built a payload bay: a coupler nested in the airframe carrying a hollow payload sphere, with a tail section abutted behind.

Setup

An empty directory with `Aerotech.rse` copied in (`scripts/tutorial3.cli` replays everything). Expect this script to exit with code 1 — it contains two intentional failures, and the exit code honestly says so.

A clean three-part stack

```
# QtRocket CLI - type 'help' for commands, 'quit' to exit.
loadmotors Aerotech.rse
OK loadmotors: 249 motors from Aerotech.rse
newdesign NoseCone name=Nose baseRadius=0.02 length=0.12 density=1400
OK newdesign: root NoseCone id=2
addpart Nose BodyTube name=Airframe innerRadius=0.0191 outerRadius=0.02 length=0.40 density=1400
OK addpart: BodyTube id=3 under 2
addpart Airframe FinSet name=Fins finCount=3 rootChord=0.06 tipChord=0.03 span=0.045 sweep=0.03
→ thickness=0.003 bodyRadius=0.02 density=1400 seat=OnSurface parentStation=0 childStation=0
OK addpart: FinSet id=4 under 3
checkdesign
OK checkdesign: 3 parts, contiguous, no overlaps
```

The same spine as Tutorial 1, at 40 mm scale: nose, airframe, fins-on-surface. `checkdesign` passes. Now we break it.

Mistake one: the coupler that pokes through

A coupler is a short tube nested inside the airframe's bore — `seat=NestInBore`. For that seat, `gap` means *insertion depth*: how far the child slides in from the parent's fore rim. Here is a 0.05 m coupler inserted only 0.02 m:

```
addpart Airframe BodyTube name=Coupler innerRadius=0.018 outerRadius=0.019 length=0.05 density=1400
→ seat=NestInBore gap=0.02
OK addpart: BodyTube id=5 under 3
checkdesign
ERR checkdesign: 1 overlap(s)
  overlap: part 5 (OD 0.019 m) intrudes into part 2 (capacity 0.015 m) by 0.004 m at z=-0.09 m
listparts
OK listparts:
  [2] NoseCone "Nose"  m=0.0703717 kg
  [3] BodyTube "Airframe" m=0.0619095 kg
  [4] FinSet "Fins"  m=0.025515 kg
  [5] BodyTube "Coupler" m=0.00813672 kg
-- composite: mass=0.165933 kg, cg_z=ERR listparts: Part::computeCompositeAt: placement solve failed --
→ part 5 (OD 0.019 m) intrudes into part 2 (capacity 0.015 m) by 0.004 m at z=-0.09 m
savedesign payload_broken.qrd
OK savedesign: payload_broken.qrd
```

Three lessons in one failure:

- The overlap message is *located and quantified*: the 0.03 m of coupler left sticking out of the bore reaches under the nose skirt at $z = -0.09$ m, where its 0.019 m outer diameter exceeds the nose's 0.015 m capacity — the reported 0.004 m intrusion is that *radial* penetration, at that axial location. Nothing vague.
- `listparts` still prints the tree — geometry exists — but the composite line degrades to an **ERR** mid-print: composite mass and CG are refused while the placement is unphysical. A broken design cannot silently produce plausible-looking numbers.
- `savedesign` still works. A `.qrd` records intent, including broken intent — which lets you save your work mid-repair, and lets the visualizer show you the problem (Fig. 4).

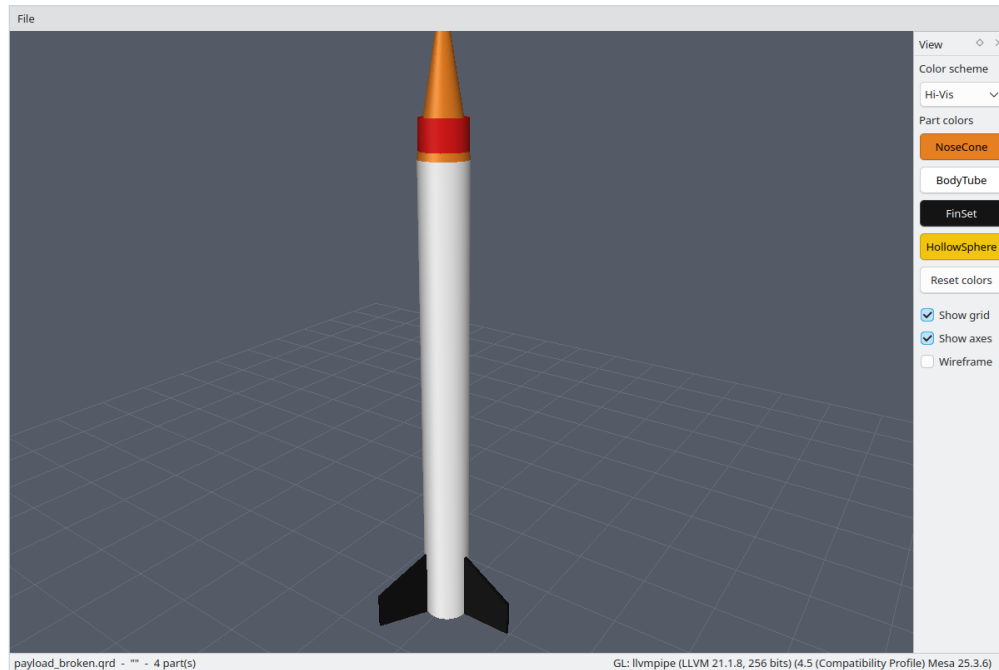


Figure 4. `payload_broken.qrd` in the visualizer (Hi-Vis scheme): the offending coupler is painted the fixed error red, jutting from the airframe's fore rim into the nose skirt. The overlap diagnostics and this red paint come from the same geometry resolver.

The fix: insertion depth equals length

```
removepart 5
OK removepart: removed id=5 (BodyTube)
addpart Airframe BodyTube name=Coupler innerRadius=0.018 outerRadius=0.019 length=0.05 density=1400
↳ seat=NestInBore gap=0.05
OK addpart: BodyTube id=6 under 3
checkdesign
OK checkdesign: 4 parts, contiguous, no overlaps
```

`gap=0.05` — the full coupler length — seats it flush in the bore. Note the fix removed the broken part *by id* (`removepart 5`); in a saved script you would prefer names (Section 2.1).

An internal payload


```

addpart Coupler HollowSphere name=Payload innerRadius=0.016 outerRadius=0.0175 density=1200 seat=NestInBore
↳ gap=0.035
OK addpart: HollowSphere id=7 under 6
checkdesign
OK checkdesign: 5 parts, contiguous, no overlaps

```

A hollow sphere, 17.5 mm outer radius, nested inside the coupler. Internal parts stack the same way external ones do: **NestInBore** into the coupler's bore, insertion depth placing it fully inside. `checkdesign` still passes — the resolver knows a sphere inside a tube inside a tube is fine, because the bore capacities say so.

Mistake two: the floating tail

Abut means *touching*: child's fore plane against parent's aft plane, `gap` a signed standoff along +z. A sign slip — `gap=-0.02` — leaves two centimeters of daylight:

```

addpart Airframe BodyTube name=Tail innerRadius=0.0191 outerRadius=0.02 length=0.06 density=1400 seat=Abut
↳ gap=-0.02
OK addpart: BodyTube id=8 under 3
checkdesign
ERR checkdesign: 1 air gap(s)
gap: nothing spans z in [-0.54, -0.52] (0.02 m)

```

An *air gap* is the other way a stack can be wrong: nothing overlaps, but the rocket is not one connected body. Same locate-and-quantify style, same honest refusal to compute a composite.

The fix, and the finished bay

```

removepart 8
OK removepart: removed id=8 (BodyTube)
addpart Airframe BodyTube name=Tail innerRadius=0.0191 outerRadius=0.02 length=0.06 density=1400 seat=Abut
OK addpart: BodyTube id=9 under 3
checkdesign
OK checkdesign: 6 parts, contiguous, no overlaps
listparts
OK listparts:
  [2] NoseCone "Nose"  m=0.0703717 kg
  [3] BodyTube "Airframe" m=0.0619095 kg
  [4] FinSet "Fins" m=0.025515 kg
  [6] BodyTube "Coupler" m=0.00813672 kg
  [7] HollowSphere "Payload" m=0.00635042 kg
  [9] BodyTube "Tail" m=0.00928642 kg
  -- composite: mass=0.18157 kg, cg_z=-0.251348 m (t=0)
setmotor I280DM
OK setmotor: I280DM
setdrag 0.75
OK setdrag: 0.75
setarea 0.001257
OK setarea: 0.001257 m^2
savedesign payload_final.qrd
OK savedesign: payload_final.qrd
launch
OK launch: 4506 steps, t_final=45.050s
apogee    = 2121.606 m @ t=14.450s
max_speed = 590.492 m/s @ t=1.750s
downrange = 0.000 m

```

```

landing = t=45.050s, pos=(0.000, 0.000, -0.483)
CSV /tmp/flight1/qtrocket_run.csv
quit
OK bye

```

Leave `gap` at its default of zero: `abut` means touching. The finished six-part design flies (that is Tutorial 4's `airframe` class) and looks — from the outside — identical to a rocket with no payload at all (Fig. 5).

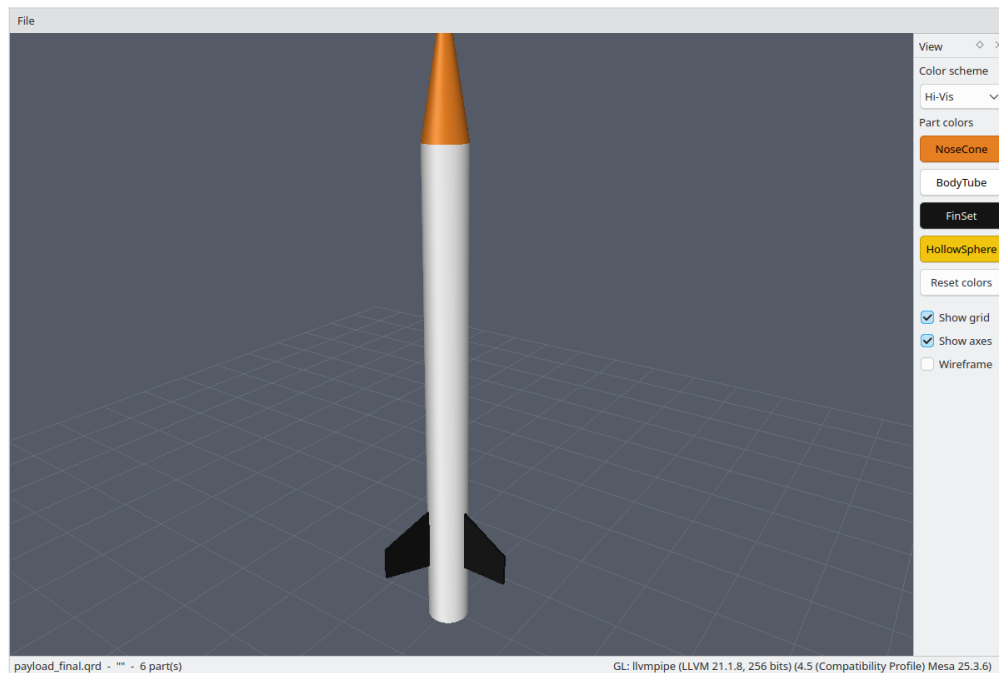


Figure 5. `payload_final.qrd`: the repaired stack. The coupler and payload sphere are fully internal — the outside gives no hint, which is precisely why `checkdesign`, not the eyeball, is the arbiter of internal geometry.

The red paint and the ERR text are the same fact

`qtrocket-visualizer` runs the *same placement resolver* the simulator uses. When `checkdesign` names an offender, the visualizer paints that part red — one geometry model, two views of its verdict. If the render looks fine but `checkdesign` objects, believe `checkdesign`: it can see inside.

What you learned

Abut/NestInBore/OnSurface and what `gap` means to each; overlap vs air-gap diagnostics, located and quantified; broken designs save but refuse composites; the visualizer's error-red; and that a script with intentional failures exits 1 — which Tutorial 5 turns into a feature.

6 Tutorial 4 — The Physics Knobs: Atmospheres, Integrators, Timestep, Angle

Six flights of one airframe, each isolating one knob: which atmosphere supplies the air, which integrator does the arithmetic, how big the timestep is, and which way the launch rail points. The airframe is Tutorial 2's 26 mm bird on a D12; if you skipped ahead, the script rebuilds it.

Setup

An empty directory with `estes_small.qmd` copied in; `scripts/tutorial4.cli` replays the whole study. Each `launch` is followed by a `save` so its trajectory survives the next flight.

The knobs, listed

```
# QtRocket CLI - type 'help' for commands, 'quit' to exit.
loaddb estes_small.qmd
OK loaddb: 13 new motors from estes_small.qmd (13 total)
newdesign NoseCone name=nose baseRadius=0.013 length=0.09 density=900
OK newdesign: root NoseCone id=2
addpart nose BodyTube name=body innerRadius=0.0121 outerRadius=0.013 length=0.30 density=1000
OK addpart: BodyTube id=3 under 2
addpart body FinSet name=fins finCount=3 rootChord=0.05 tipChord=0.025 span=0.03 sweep=0.02 thickness=0.003
↪ bodyRadius=0.013 density=900 seat=OnSurface parentStation=0 childStation=0
OK addpart: FinSet id=4 under 3
checkdesign
OK checkdesign: 3 parts, contiguous, no overlaps
setmotor D12
OK setmotor: D12
setdrag 0.75
OK setdrag: 0.75
setarea 0.000531
OK setarea: 0.000531 m^2
listatmospheres
OK listatmospheres: 3 available
Constant Atmosphere
US Standard 1976
Vacuum
listgravity
OK listgravity: 2 available
Constant Gravity
Spherical Gravity
listintegrators
OK listintegrators: 2 available
Runge-Kutta 4th Order
Runge-Kutta-Fehlberg
```

Three `list...` commands enumerate what is pluggable: three atmosphere models, two gravity models, two integrators. `setatmosphere`, `setgravity`, and `setintegrator` accept exactly these names — misspell one and the CLI rejects it and keeps the current model.

Baseline: constant air, RK4, $\Delta t = 0.01$ s

```
set timestep 0.01
OK set timestep: 0.01 s
status
```

```
OK status:
  motor      = D12
  dry_mass   = 1 kg
  drag_coeff = 0.75
  ref_area   = 0.000531 m^2
  velocity   = 0 m/s
  angle      = 0 deg (from vertical)
  atmosphere = Constant Atmosphere
  gravity     = Constant Gravity
  integrator = Runge-Kutta 4th Order
  database   = 13 motors
launch
OK launch: 2037 steps, t_final=20.360s
  apogee     = 447.674 m @ t=8.140s
  max_speed  = 145.186 m/s @ t=1.590s
  downrange  = 0.000 m
  landing    = t=20.360s, pos=(0.000, 0.000, -0.350)
CSV /tmp/flight1/qtrocket_run.csv
save baseline_constant.csv
OK save: /tmp/flight1/baseline_constant.csv
```

447.7 m. Drag is computed from the *active atmosphere's* density at the rocket's current altitude — that is the whole meaning of the atmosphere setting. `Constant Atmosphere` uses sea-level density everywhere.

Real air: US Standard 1976

```
setatmosphere US Standard 1976
OK setatmosphere: US Standard 1976
launch
OK launch: 2048 steps, t_final=20.470s
  apogee     = 452.936 m @ t=8.210s
  max_speed  = 145.514 m/s @ t=1.600s
  downrange  = 0.000 m
  landing    = t=20.470s, pos=(0.000, 0.000, -0.226)
CSV /tmp/flight1/qtrocket_run.csv
save us_standard.csv
OK save: /tmp/flight1/us_standard.csv
```

452.9 m — about one percent higher, because air thins with altitude and a 450-meter flight barely samples the difference. Model fidelity buys little on low flights and progressively more as you climb; knowing *when* a better model matters is as useful as having it.

No air at all: the vacuum ceiling

```
setatmosphere Vacuum
OK setatmosphere: Vacuum
launch
OK launch: 4338 steps, t_final=43.370s
  apogee     = 2223.647 m @ t=22.070s
  max_speed  = 208.924 m/s @ t=43.370s
  downrange  = 0.000 m
  landing    = t=43.370s, pos=(0.000, 0.000, -1.841)
CSV /tmp/flight1/qtrocket_run.csv
save vacuum_rk4.csv
OK save: /tmp/flight1/vacuum_rk4.csv
```

2223.6 m — five times the apogee, from the same motor. Vacuum reduces the model to thrust plus gravity, which makes it two useful things: a deterministic upper bound (“what is drag costing me?” — here, 80% of the theoretical altitude) and a diagnostic (if a flight surprises you, fly it in vacuum to separate propulsion questions from aerodynamics questions).

Same flight, adaptive integrator

```
setintegrator Runge-Kutta-Fehlberg
OK setintegrator: Runge-Kutta-Fehlberg
launch
OK launch: 528 steps, t_final=43.290s
  apogee    = 2223.530 m @ t=21.990s
  max_speed = 209.023 m/s @ t=43.290s
  downrange = 0.000 m
  landing   = t=43.290s, pos=(0.000, 0.000, -4.080)
CSV /tmp/flight1/qtrocket_run.csv
save vacuum_rkf45.csv
OK save: /tmp/flight1/vacuum_rkf45.csv
```

The Runge–Kutta–Fehlberg integrator adapts its own step size against an error estimate. Its vacuum apogee agrees with fixed-step RK4 to within 0.12 m in 2223 — five parts in a hundred thousand — while choosing its own steps. Agreement between two independent integration schemes is the cheapest confidence check numerical work offers.

What a coarse timestep costs

```
setintegrator Runge-Kutta 4th Order
OK setintegrator: Runge-Kutta 4th Order
set timestep 0.1
OK set timestep: 0.1 s
launch
OK launch: 430 steps, t_final=42.900s
  apogee    = 2183.363 m @ t=21.800s
  max_speed = 207.113 m/s @ t=42.900s
  downrange = 0.000 m
  landing   = t=42.900s, pos=(0.000, 0.000, -3.702)
CSV /tmp/flight1/qtrocket_run.csv
save coarse_dt.csv
OK save: /tmp/flight1/coarse_dt.csv
set timestep 0.01
OK set timestep: 0.01 s
```

Back on RK4 with $\Delta t = 0.1$ s — ten times coarser — the same vacuum flight reads 2183.4 m: forty meters (1.8%) short, lost to integration error across the sharp thrust transient. The lesson generalizes: when a result matters, halve the timestep and see if the answer moves.

Tilt the rail

```
setatmosphere US Standard 1976
OK setatmosphere: US Standard 1976
setangle 10
OK setangle: 10 deg
setvelocity 15
OK setvelocity: 15 m/s
launch
```

```

OK launch: 2106 steps, t_final=21.050s
  apogee    = 479.803 m @ t=8.280s
  max_speed = 151.030 m/s @ t=1.590s
  downrange = 12.127 m
  landing   = t=21.050s, pos=(12.127, 0.000, -0.118)
CSV /tmp/flight1/qtrocket_run.csv
save angled.csv
OK save: /tmp/flight1/angled.csv
quit
OK bye

```

Two changes together: a 15 m/s rail-exit velocity (`setvelocity`) and a 10° tilt from vertical (`setangle`). For the first time `downrange` is nonzero — 12.1 m of drift — and apogee ticks up relative to the upright US-Standard flight because the rail speed adds kinetic energy. Figure 6 shows both studies: the three atmosphere curves overlaid, and the tilted flight's ground track.

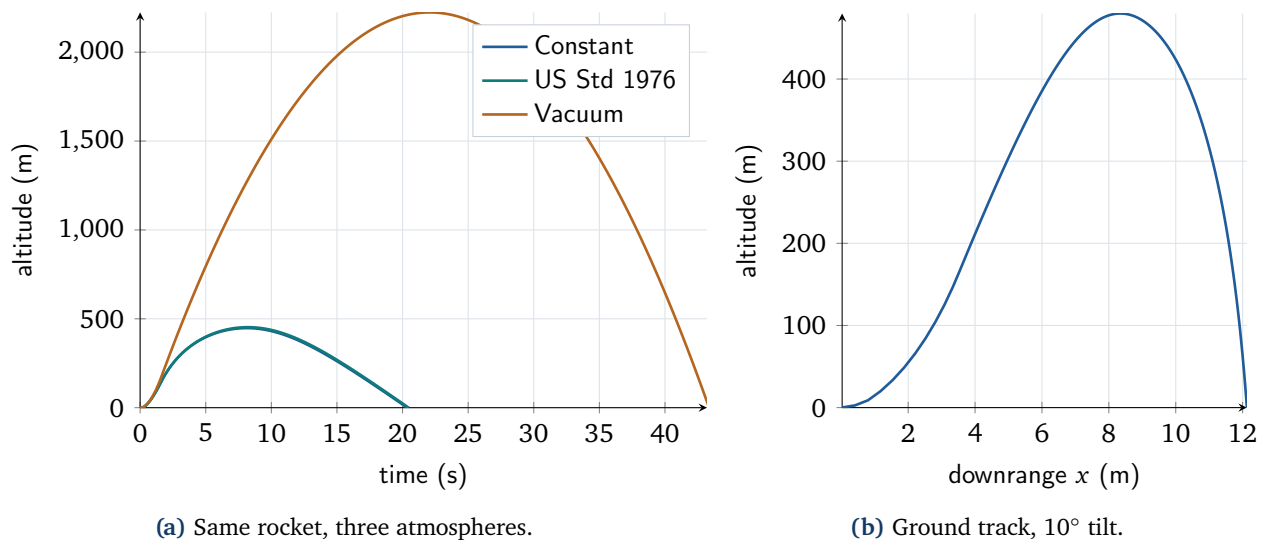


Figure 6. Tutorial 4's studies, plotted from the `saved` CSVs: the atmosphere overlay (left) and the tilted launch's x - z trajectory (right).

What you learned

Atmosphere selection is $\rho(z)$ selection; vacuum is a bound and a diagnostic; two integrators agreeing is cheap confidence; coarse timesteps have a measurable price; and the launch rail's speed and angle are the last two `status` rows you had not touched yet.

7 Tutorial 5 — One-Button Flights: Scripts, Exit Codes, and CSV

The capstone is short because you already know all the commands — this tutorial is about turning them into automation: script files, one-liners, exit codes a shell can branch on, the save/load round trip, and the trajectory CSV.

Build once, save, prove the round trip

```
# QtRocket CLI - type 'help' for commands, 'quit' to exit.
loaddb estes_small.qmd
OK loaddb: 13 new motors from estes_small.qmd (13 total)
newdesign NoseCone name=nose baseRadius=0.0098 length=0.10 density=700
OK newdesign: root NoseCone id=2
addpart nose BodyTube name=body innerRadius=0.0091 outerRadius=0.0098 length=0.25 density=700
OK addpart: BodyTube id=3 under 2
addpart body FinSet name=fins finCount=3 rootChord=0.04 tipChord=0.02 span=0.03 sweep=0.02 thickness=0.003
↪ bodyRadius=0.0098 density=700 seat=OnSurface parentStation=0 childStation=0
OK addpart: FinSet id=4 under 3
setmotor C6
OK setmotor: C6
setdrag 0.75
OK setdrag: 0.75
setarea 0.000302
OK setarea: 0.000302 m^2
checkdesign
OK checkdesign: 4 parts, contiguous, no overlaps
savedesign workhorse.qrd
OK savedesign: workhorse.qrd

cleardesign
OK cleardesign: restored the placeholder body
loaddesign workhorse.qrd
OK loaddesign: workhorse.qrd (motor C6)
checkdesign
OK checkdesign: 4 parts, contiguous, no overlaps
status
OK status:
  motor      = C6
  dry_mass   = 1 kg
  drag_coeff  = 0.75
  ref_area   = 0.000302 m^2
  velocity   = 0 m/s
  angle      = 0 deg (from vertical)
  atmosphere = Constant Atmosphere
  gravity     = Constant Gravity
  integrator  = Runge-Kutta 4th Order
  database    = 13 motors
launch
OK launch: 2041 steps, t_final=20.400s
  apogee     = 436.562 m @ t=7.990s
  max_speed  = 143.732 m/s @ t=1.820s
  downrange  = 0.000 m
  landing    = t=20.400s, pos=(0.000, 0.000, -0.436)
CSV /tmp/flight1/qtrocket_run.csv
```

`cleardesign` wipes the session; `loaddesign` restores everything from the `.qrd` — tree, placement, drag configuration, and the motor re-resolved by *name* from the loaded database. The reloaded

rocket passes the same `checkdesign` and flies the same flight. That is the round-trip contract, and it is what makes design files trustworthy carriers of work between sessions (and between the CLI, the GUI, and the visualizer).

Sampling the trajectory

```
states 200
OK states: 2041 total, stride=200
t,x,y,z,vx,vy,vz,mass,cg_x,cg_y,cg_z,Ixx,Iyy,Izz
0,0,0,0,0,0,0,0.0440836721,0,0,-0.131975423,0.000430023519,0.000430023519,5.27738786e-06
2,0,0,185.449796,0,0,129.355482,0.0332836721,0,0,-0.150463003,0.000378959825,0.000378959825,4.83998786e-06
4,0,0,347.944129,0,0,50.7177043,0.0332836721,0,0,-0.150463003,0.000378959825,0.000378959825,4.83998786e-06
6,0,0,416.493841,0,0,20.6988221,0.0332836721,0,0,-0.150463003,0.000378959825,0.000378959825,4.83998786e-06
8,0,0,436.561845,0,0,-0.0492822118,0.0332836721,0,0,-0.150463003,0.000378959825,0.000378959825,4.83998786e-06
↪ e-06
10,0,0,417.367307,0,0,-18.6519424,0.0332836721,0,0,-0.150463003,0.000378959825,0.000378959825,4.83998786e-06
↪ -06
12,0,0,365.250398,0,0,-32.4712673,0.0332836721,0,0,-0.150463003,0.000378959825,0.000378959825,4.83998786e-06
↪ -06
14,0,0,291.30428,0,0,-40.6424912,0.0332836721,0,0,-0.150463003,0.000378959825,0.000378959825,4.83998786e-06
↪ 06
16,0,0,205.327326,0,0,-44.8380887,0.0332836721,0,0,-0.150463003,0.000378959825,0.000378959825,4.83998786e-06
↪ -06
18,0,0,113.399757,0,0,-46.8368018,0.0332836721,0,0,-0.150463003,0.000378959825,0.000378959825,4.83998786e-06
↪ -06
20,0,0,18.6884925,0,0,-47.7549149,0.0332836721,0,0,-0.150463003,0.000378959825,0.000378959825,4.83998786e-06
↪ -06
save workhorse_flight.csv
OK save: /tmp/flight1/workhorse_flight.csv
quit
OK bye
```

`states` prints every n -th state sample to the terminal — the same full CSV rows `save` writes, all fourteen columns — for a quick look without leaving the session; `save` writes the complete series to a file. The columns are self-describing:

```
t,x,y,z,vx,vy,vz,mass,cg_x,cg_y,cg_z,Ixx,Iyy,Izz
```

— time, position, velocity, plus the composite mass properties at every step (mass falls during the burn; the inertia columns are what a future 6-DOF mode will consume). Every plot in this guide is drawn from these files.

One-liners and exit codes

Real captures, dollar-prompts and all:

```
$ qtrocket-cli -c "loaddb estes_small.qmd" ; echo $?
# QtRocket CLI - type 'help' for commands, 'quit' to exit.
OK loaddb: 13 new motors from estes_small.qmd (13 total)
0
$ qtrocket-cli -c "launch" ; echo $?
# QtRocket CLI - type 'help' for commands, 'quit' to exit.
ERR launch: no motor set (use loadmotors + setmotor first)
1
```



```
$ qtrocket-cli tutorial5-missing.cli ; echo $?  
error: cannot open script file 'tutorial5-missing.cli'  
2
```

The middle example is the process-isolation warning from [Section 1.2](#) made concrete: each `-c` is a fresh session, so the launch never saw a motor. The exit-code contract — 0 all-OK, 1 any **ERR**, 2 bad invocation — makes the CLI an honest citizen of shell scripts and CI pipelines:

```
qtrocket-cli nightly_designs.cli || echo "a design regressed"
```

What you learned — and where to go next

Scripts are just sessions in a file; `.qrd` files carry designs across sessions and tools; the CSV carries flights into your plotting tool of choice; and the exit code turns any script into a regression guard — Tutorial 3's deliberate failures exited 1 for exactly this reason. From here: the command reference ([Section 8](#)) is the complete vocabulary, and the design whitepapers under [docs/](#) go as deep as you care to dig.

8 Command reference

Every command answers with a line beginning **OK** or **ERR**; detail lines usually follow it, with two exceptions: `help`'s #-prefixed listing precedes its terminating `OK help`, and comment or blank lines produce no output at all. The full protocol is described in [Section 1.3](#). Commands are grouped thematically below; the source of truth is `cli/Repl.cpp`.

8.1 Session and meta commands

`help`

Prints the command list as #-prefixed comment lines (one line per command with a short description, plus notes: link-key summary, 'part ids reset on reload; launch/atmosphere settings are session config and are NOT saved in the design file', and 'drag uses the active atmosphere's density; select the Vacuum atmosphere to reduce the model to thrust + gravity.').

Arguments. None. Extra tokens are ignored.

On success. A block of lines each beginning '# ' (starting '# commands:'), followed by the terminating line 'OK help'.

Errors. None.

Side effects. None.

`quit (alias: exit)`

Leaves the REPL. The only command whose handler returns false, ending `Repl::run`'s read loop.

Arguments. None.

On success. OK bye

Errors. None.

Side effects. Terminates the session; `run()` then returns exit code 0 or 1 based on accumulated ERR count.

`# any text | (empty line)`

A blank/whitespace-only line is ignored silently. A line whose first token starts with '#' (either '# text' or '#text') is a comment and ignored silently. Both keep the session going and produce no output at all (no OK line).

Arguments. N/A

On success. (nothing)

Errors. None.

Side effects. None.

`<anything not recognized>`

Fallthrough when the lowercased command word matches nothing.

Arguments. N/A

On success. N/A

Errors. ERR unknown command '<cmd>' (try help) — <cmd> is the lowercased command word.

Side effects. Counts toward the ERR exit code.

8.2 Motor database commands

loadmotors <file.rse|file.eng>

Imports a local motor file into the in-memory motor database (merges with whatever is already loaded). Format is chosen by extension, lowercased: '.eng' goes to MotorModelDatabase::importRASPPFile (RASP), any other extension goes to importRSEFile (RockSim XML). No network.

Arguments. path (required): rest of the line after the command word, trimmed — may contain spaces. Case preserved.

On success. OK loadmotors: <added> motors from <path>

Errors. Empty path: 'ERR usage: loadmotors <file.rse|file.eng>'. Any exception from the importer: 'ERR loadmotors: <what()>'.

Side effects. Adds motors to the session MotorModelDatabase. Session-only until savedb.

savedb <file.qmd>

Saves the entire in-memory motor database to QtRocket's own XML motor-database format (.qmd) via MotorModelDatabase::saveMotorDatabase (Boost property_tree XML). No network.

Arguments. path (required): rest-of-line, trimmed, may contain spaces.

On success. OK savedb: <db-size> motors to <path>

Errors. Empty path: 'ERR usage: savedb <file.qmd>'. Exception while saving: 'ERR savedb: <what()>'.

Side effects. Writes the file at <path> (as given; not made absolute).

loadddb <file.qmd>

Loads a previously saved .qmd motor database, merging it into (adding to) the current in-memory database. No network.

Arguments. path (required): rest-of-line, trimmed.

On success. OK loadddb: <new> new motors from <path> (<total> total) — <new> = db size after minus before; <total> = size after.

Errors. Empty path: 'ERR usage: loadddb <file.qmd>'. Exception while loading: 'ERR loadddb: <what()>'.

Side effects. Adds motors to the session database.

tcfacets

Fetches the thrustcurve.org search facets (valid manufacturers, diameters, impulse classes) via the online REST client. Requires network.

Arguments. None.

On success. 'OK tcfacets:' then three labeled lines with space-separated values: ' manufacturers: <m> <m> ...', ' diameters: <d> <d> ...', ' impulseClasses: <c> <c> ...'.

Errors. Any exception (e.g. network failure): 'ERR tcfacets: <what()>'.

Side effects. None (read-only query).

tcsearch [manufacturer=<str>] [diameter=<mm>] [impulseClass=<str>]

Searches thrustcurve.org (requires network) and merges every result into the in-memory motor database so setmotor/listmotors can see them. Recognized keys: manufacturer, impulseClass (strings, case preserved), diameter (double). Tokens without '=' are silently skipped; unknown keys with '=' are silently ignored. Note: diameter uses plain std::stod, NOT the strict full-token parser — a value with a numeric prefix and trailing garbage (e.g. '18mm') is accepted as 18; only a fully non-numeric value errors.

Arguments. Zero or more key=value tokens; all optional (an unconstrained search is allowed). diameter: double via std::stod (lenient); manufacturer/impulseClass: raw strings.

On success. 'OK tcsearch: <n> motors' then one line per result: '<commonName> avg=<avgThrust>N Itot=<totalImpulse>Ns'.

Errors. Non-numeric diameter: 'ERR tcsearch: bad diameter '<val>'. Any exception from the online search: 'ERR tcsearch: <what()>'.

Side effects. Found motors are added to the session motor database.

listmotors [substr]

Lists motor common names in the database, sorted by name (the map is keyed by commonName). The optional filter is the whole rest of the line (may contain spaces) used as a case-sensitive substring match against commonName (MotorQuery::nameContains, std::string::find).

Arguments. substr (optional): rest-of-line, trimmed, case-sensitive.

On success. 'OK listmotors: <n> shown' plus ' (filter="<substr>")' when a filter was given, then one line per motor: '<commonName> avg=<avgThrust>N Itot=<totalImpulse>Ns'.

Errors. Empty database (checked first): 'ERR listmotors: no motors loaded (use loadmotors)'.

Side effects. None.

setmotor <code>

Selects a motor by its exact common name (e.g. 'A8'); the name is the rest of the line so it may contain spaces, and lookup is an exact case-sensitive map find on commonName. Attaches the motor to the rocket via RocketModel::setMotorModel and stages motorSet/motorName for status and launch.

Arguments. code (required): rest-of-line, trimmed, case-sensitive exact common name.

On success. OK setmotor: <name>

Errors. Empty database (checked before usage): 'ERR setmotor: no motors loaded (use loadmotors)'. Empty name: 'ERR usage: setmotor <code>'. Unknown name: 'ERR setmotor: '<name>' not found (use listmotors)'.

Side effects. Sets the model's motor; session state. The motor's common name IS persisted by savedesign (.qrd stores motor-by-name); newdesign and cleardesign drop the selection.

8.3 Flight configuration commands

setmass <kg>

Sets the structural (dry) mass in kilograms. Strict numeric parse: the whole token must be a valid double ('0.5abc' is rejected, not truncated).

Arguments. kg (required): double, strict full-token parse; must be > 0. Staged default 1.0 kg.

On success. OK setmass: <m> kg

Errors. Missing/non-numeric token: 'ERR usage: setmass <kg>'. m <= 0: 'ERR setmass: mass must be > 0'.

Side effects. Calls RocketModel::setMass immediately and stages dryMass for status. Session-only; not saved in .qrd, and status's dry_mass never re-syncs from a loaded design.

setdrag <cd>

Sets the (dimensionless) drag coefficient. No range validation — any double std::stod accepts is taken, including negative, inf, and nan.

Arguments. cd (required): double, strict full-token parse. Staged default 1.0.

On success. OK setdrag: <d>

Errors. Missing/non-numeric: 'ERR usage: setdrag <cd>'.

Side effects. Calls RocketModel::setDragCoefficient immediately; stages dragCoeff. Persisted in .qrd by savedesign (aero options) and re-synced by loaddesign.

setarea <m^2>

Sets the aerodynamic reference area in square meters. Zero is allowed (disables drag area).

Arguments. m^2 (required): double, strict full-token parse; must be >= 0. Staged default 1.134e-3 m^2 (matches RocketModel's 38 mm tube default).

On success. OK setarea: <a> m^2

Errors. Missing/non-numeric: 'ERR usage: setarea <m^2>'. a < 0: 'ERR setarea: area must be >= 0'.

Side effects. Calls RocketModel::setReferenceArea immediately; stages referenceArea. Persisted in .qrd and re-synced by loaddesign.

setvelocity <m/s>

Sets the initial speed applied at launch (split into vx/vz by the launch angle). No range check. Useful as a launch-rail velocity to avoid the no-liftoff pad-hold abort on low-thrust motors.

Arguments. m/s (required): double, strict full-token parse. Staged default 0.

On success. OK setvelocity: <v> m/s

Errors. Missing/non-numeric: 'ERR usage: setvelocity <m/s>'.

Side effects. Staged only; applied when launch builds the initial state. Session-only, never saved in .qrd.

setangle <deg>

Sets the launch angle in degrees from vertical (0 = straight up, 90 = horizontal). No range check. At launch: vx = v*sin(angle), vz = v*cos(angle), with deg->rad via the constant DEG_PER_RAD = 57.2958.

Arguments. deg (required): double, strict full-token parse. Staged default 0.

On success. OK setangle: <a> deg

Errors. Missing/non-numeric: 'ERR usage: setangle <deg>'.

Side effects. Staged only; applied at launch. Session-only.

setimestep <s>

Sets the integrator timestep in seconds (applied to the sim immediately via `QtRocket::setTimeStep`).

Arguments. *s* (required): double, strict full-token parse; must be > 0 .

On success. OK setimestep: <dt> *s*

Errors. Missing/non-numeric: 'ERR usage: setimestep <seconds>'. $dt \leq 0$: 'ERR setimestep: must be > 0 '.

Side effects. Changes the propagator timestep. Session-only; not shown in status, not saved in .qrd.

8.4 Environment and numerics commands

listatmospheres

Lists the available atmosphere model names. The current set (`Environment.h`): 'Constant Atmosphere', 'US Standard 1976', 'Vacuum'.

Arguments. None.

On success. 'OK listatmospheres: <n> available' then one model name per line (unprefixed).

Errors. None.

Side effects. None.

setatmosphere <name>

Selects the atmosphere model. Name is the rest of the line (may contain spaces, e.g. 'US Standard 1976'), case-sensitive, validated by exact match against `getAvailableAtmosphereModels()`. Default 'Constant Atmosphere'. Selecting 'Vacuum' removes drag entirely (thrust + gravity only).

Arguments. *name* (required): rest-of-line, trimmed, case-sensitive; must be one of the listatmospheres names.

On success. OK setatmosphere: <name>

Errors. Empty: 'ERR usage: setatmosphere <name> (see listatmospheres)' (two spaces before the parenthesis). Not in list: 'ERR setatmosphere: '<name>' not found (see listatmospheres)'.

Side effects. Sets Environment's atmosphere model immediately; stages the name for status. Session-only, not saved in .qrd.

listgravity

Lists the available gravity model names. Current set (`Environment.h`): 'Constant Gravity', 'Spherical Gravity'.

Arguments. None.

On success. 'OK listgravity: <n> available' then one model name per line.

Errors. None.

Side effects. None.

setgravity <name>

Selects the gravity model; rest-of-line name, case-sensitive, validated against `getAvailableGravityModels()`. Default 'Constant Gravity'.

Arguments. *name* (required): rest-of-line, trimmed, case-sensitive.

On success. OK setgravity: <name>

Errors. Empty: 'ERR usage: setgravity <name> (see listgravity)'. Not in list: 'ERR setgravity: '<name>' not found (see listgravity)'.

Side effects. Sets Environment's gravity model immediately; stages for status. Session-only.

listintegrators

Lists the available integrator names (from a temporary sim::Integrator). Current set (Integrator.h): 'Runge-Kutta 4th Order' (fixed step), 'Runge-Kutta-Fehlberg' (adaptive RK45).

Arguments. None.

On success. 'OK listintegrators: <n> available' then one name per line.

Errors. None.

Side effects. None.

setintegrator <name>

Selects the integrator; rest-of-line name, case-sensitive, validated against getAvailableIntegratorModels(). Default 'Runge-Kutta 4th Order'.

Arguments. name (required): rest-of-line, trimmed, case-sensitive.

On success. OK setintegrator: <name>

Errors. Empty: 'ERR usage: setintegrator <name> (see listintegrators)'. Not in list: 'ERR setintegrator: '<name>' not found (see listintegrators)'.

Side effects. Calls QtRocket::setIntegratorModel; stages for status. Session-only.

8.5 Design commands

newdesign <type> [name=..] [geom key=value...]

Starts a new design with a root part built by the part factory (types: NoseCone, BodyTube, FinSet, HollowSphere — case-sensitive; see addpart grammar). The remaining tokens use exactly the same key=value grammar as addpart; link tokens (seat/parentStation/childStation/gap) are parsed uniformly but ignored, since a root has no parent link. Replaces the entire existing design via setRoot.

Arguments. type (required, positional, case-sensitive factory key); then zero or more key=value tokens per the addpart grammar.

On success. OK newdesign: root <type> id=<id>

Errors. Missing type: 'ERR usage: newdesign <type> [name=..] [geom key=value...]'. Token/value parse error: 'ERR newdesign: <parse error>' (e.g. "expected key=value, got '<tok>'", "bad value for '<key>': '<val>'"). Factory throw (missing required field / unknown type / non-physical geometry from the part ctor): 'ERR newdesign: <what()>' e.g. "makePart: BodyTube requires 'outerRadius'".

Side effects. setRoot replaces the whole part tree AND drops any previously-set motor: staged motorSet is cleared and motorName emptied, so setmotor must be re-run before launch.

cleardesign

Resets the rocket to the default placeholder body (RocketModel::clearDesign).

Arguments. None.

On success. OK cleardesign: restored the placeholder body

Errors. None.

Side effects. Discards the part tree; clears the staged motor (motorSet=false, motorName emptied).

addpart <parentId|name|root> <type> [name=..] [geom key=value...] [seat=.. parentStation=.. childStation=.. gap=..]

Attaches a factory-built part under an existing parent. Parent addressing, tried in order: the literal token 'root' (exact lowercase match — arguments keep case, so 'Root' falls through to name lookup) selects the top part; a token that parses fully as an unsigned integer is treated as a part id; otherwise it is a part name, resolved to the FIRST match in depth-first attachment order (names need not be unique; ids are session-specific and reassigned on reload, so scripts should address by unique name). Unknown key=value keys are skipped with a logger warning ('ignoring unknown design key '<key>'), which the CLI's ERROR-pinned logger suppresses entirely — the command still succeeds.

Arguments. parent (required positional): id | name | 'root'. type (required positional): NoseCone | BodyTube | FinSet | HollowSphere (case-sensitive). Then key=value tokens: geometry doubles strict-parsed; finCount strict unsigned; name string; solid boolean ('true' or '1' = true, anything else false); link keys seat/parentStation/childStation/gap. Caveat: std::stoull/stoul accept a leading '-' and wrap modulo 2^{64} , so a negative id parses to a huge value and reports 'no part with id ...' rather than a bad-id error.

On success. OK addpart: <type> id=<childId> under <parentId>

Errors. Missing positionals: 'ERR usage: addpart <parentId|name|root> <type> [name=..] [geom key=value...] [seat=.. parentStation=.. childStation=.. gap=..]'. No design: 'ERR addpart: no design (use newdesign first)'. Parent not found by name: 'ERR addpart: no part named '<tok>' (use an id, a name, or 'root')'. Token parse: 'ERR addpart: expected key=value, got '<tok>' or 'ERR addpart: bad value for '<key>': '<val>'. Link errors: 'ERR addpart: unknown seat '<s>' (Abut, NestInBore, OnSurface)' and 'ERR addpart: NestInBore gap is an insertion depth and must be ≥ 0 '. Factory throw: 'ERR addpart: <what()>'. Attach rejected / bad numeric id: 'ERR addpart: no part with id <parentId> (or the attach was rejected)'.

Side effects. Mutates the part tree (child added by move). Persisted by savedesign.

checkdesign

Physical-sense check in two layers: (1) the cached placement-solve diagnostics (radial overlaps / poke-through) from Part::placementDiagnostics(); (2) an axial-coverage scan over resolved placements for air gaps — sorted spans of positive-length parts (zero-length parts like a Motor cannot bound coverage), tolerance $1e-9$ m. A positive standoff is legal in a StationLink but leaves parts floating apart, which this reports.

Arguments. None.

On success. OK checkdesign: <n> parts, contiguous, no overlaps

Errors. No design: 'ERR checkdesign: no design'. Problems found: 'ERR checkdesign: ' + ' <n> overlap(s)' if the solve failed and/or ', <n> air gap(s)' (comma only when both), then detail lines 'overlap: <message>' per diagnostic and 'gap: nothing spans z in [<lo>, <hi>] (<len> m)' per gap.

Side effects. None.

listparts

Prints the part tree, one part per line indented two spaces per depth level, then the composite mass/CG at $t=0$. Note: the composite read can throw on a design whose placement solve failed; the top-level guard converts that to 'ERR listparts: <what()>' and the session survives.

Arguments. None.

On success. 'OK listparts:' then per part ' [id] <typeName> "<name>" m=<own mass at $t=0$ > kg' (extra 2-space indent per depth), then ' - composite: mass=<kg> kg, cg_z=<z> m ($t=0$)'.

Errors. No design: 'ERR listparts: no design'. Failed placement solve: 'ERR listparts: <exception what()>' via the catch-all. On the exception path the command word is echoed as typed (not lowercased): 'ERR <cmd-as-typed>: <what()>'.

Side effects. None (may trigger the lazy composite recompute).

removepart <id>

Removes a part and its entire sub-tree, addressed by numeric id only (see listparts for ids; ids are session-specific). The root cannot be removed.

Arguments. id (required): unsigned integer, strict full-token parse (stoull, trailing garbage rejected).

On success. OK removepart: removed id=<id> (<typeName>)

Errors. Missing token: 'ERR usage: removepart <id>'. Non-integer: 'ERR removepart: bad id '<tok>'. Root id: 'ERR removepart: cannot remove the root (use newdesign or cleardesign)'. Unknown id: 'ERR removepart: no part with id <id>'.

Side effects. Detaches the sub-tree. If the motor was inside the removed sub-tree (rocket->isMotorSet() now false), the staged motorSet flag is cleared.

savedesign <file.qrd>

Saves the rocket design via DesignSerializer::save. Persisted in the .qrd: the part tree (geometry, names, placement links), the motor BY COMMON NAME, and the aero options (drag coefficient, reference area). NOT persisted (session-only): initial velocity, launch angle, timestep, atmosphere/-gravity/integrator selections, the setmass dry-mass override, and part ids (ids are reassigned on reload).

Arguments. path (required): rest-of-line, trimmed.

On success. OK savedesign: <path>

Errors. Empty path: 'ERR usage: savedesign <file.qrd>'. Exception: 'ERR savedesign: <what()>'.

Side effects. Writes the .qrd file.

loaddesign <file.qrd>

Loads a design in place via DesignSerializer::load, re-resolving the motor by common name against the CURRENT motor database — load the motor db first (loaddb/loadmotors) or the design comes back with no motor. This is the only command that re-syncs staged config from the model: dragCoeff and referenceArea are read back from the loaded rocket, and motorSet/motorName reflect whether the by-name motor resolve succeeded. dry_mass is NOT re-synced. Part ids are reassigned on load.

Arguments. path (required): rest-of-line, trimmed.

On success. 'OK loaddesign: <path> (motor <name>)' when the motor resolved, else 'OK loaddesign: <path> (no motor)'.

Errors. Empty path: 'ERR usage: loaddesign <file.qrd>'. Exception: 'ERR loaddesign: <what()>'.

Side effects. Replaces the part tree (setRoot); may attach a motor; updates staged dragCoeff/referenceArea/motorSet/motorName.

8.6 Run and inspection commands

status

Shows the Repl's STAGED configuration, not the live model. Caveat: dragCoeff/refArea/motor re-sync from the model only inside loaddesign's handler; dry_mass NEVER re-syncs (a loaded design's composite mass is not reflected — status keeps showing the last setmass value or the 1.0 kg default). Defaults shown on a fresh session: motor (none), dry_mass 1 kg, drag_coeff 1, ref_area 0.001134 m², velocity 0, angle 0, atmosphere 'Constant Atmosphere', gravity 'Constant Gravity', integrator 'Runge-Kutta 4th Order', database (none loaded).

Arguments. None.

On success. 'OK status:' then exactly these labeled lines: ' motor = <name|(none)>', ' dry_mass = <v> kg', ' drag_coeff = <v>', ' ref_area = <v> m^2', ' velocity = <v> m/s', ' angle = <v> deg (from vertical)', ' atmosphere = <name>', ' gravity = <name>', ' integrator = <name>', ' database = <N motors|(none loaded)>'.

Errors. None.

Side effects. None.

launch

Runs the simulation to termination (altitude $z < 0$ nominally). Builds the initial state from the staged velocity/angle: position (0,0,0), velocity ($v \cdot \sin(a)$, 0, $v \cdot \cos(a)$) with a in radians via DEG_PER_RAD=57.2958; calls setInitialState + launchRocket. Pad-hold: the Propagator clamps the rocket to the pad ($z=0$, velocity zeroed) until a step genuinely lifts it, and aborts as NoLiftoff if it hasn't cleared noLiftoffAltitude=1.0 m within noLiftoffTime=3.0 s (Propagator.h 26-27) — hence the launch-rail setvelocity workaround for low-thrust motors. Apogee/max-speed come from live TrajectoryStatistics; downrange = hypot(x,y) of the final sample. Termination reason texts (WARN/abort messages): 'nominal'; 'no liftoff: never cleared 1 m after 3 s (check thrust vs weight)'; 'non-finite state: NaN/Inf in the trajectory'; 'max simulation time / iteration cap exceeded (flight never descended)'; 'integrator error'; fallback 'unknown'.

Arguments. None.

On success. Report at std::fixed 3 decimals: 'OK launch: <N> steps, t_final=<t>s' then ' apogee = <m> m @ t=<t>s', ' max_speed = <v> m/s @ t=<t>s', ' downrange = <m> m', ' landing = t=<t>s, pos=<x>, <y>, <z>''. If termination != Nominal, next line 'WARN: run aborted (<reason text>)''. If the CSV was written, final line 'CSV <absolute path>'.

Errors. No motor staged: 'ERR launch: no motor set (use loadmotors + setmotor first)'. Empty state series: 'ERR launch: no states produced' plus ' (run aborted: <reason text>)' appended when the reason is non-Nominal. CSV open failure: 'ERR launch: cannot open <abs path> for writing' (the report still prints; no CSV line follows).

Side effects. Writes the full state series (stride 1) to fs::absolute("qtrocket_run.csv") in the current working directory, overwriting any previous run. Replaces the stored state history read by

states/save.

states [stride]

Prints the last run's state series as CSV to stdout, keeping every stride-th sample. The stride argument is optional and forgiving: it is parsed with the strict token parser but a parse FAILURE is ignored (missing or non-numeric token both leave stride at 1 — no error). Fractional values truncate to int; values < 1 clamp to 1 via std::max.

Arguments. stride (optional, default 1): double token, truncated to int, clamped to >= 1; garbage silently treated as 1.

On success. 'OK states: <total> total, stride=<stride>' then the CSV: header 't,x,y,z,vx,vy,vz,mass,cg_x,cg_y,cg_z,Ixx,Iyy,Izz' and one row per kept sample at setprecision(9).

Errors. No prior run: 'ERR states: nothing to show (launch first)'.

Side effects. None.

save <path.csv>

Writes the last run's FULL series (stride 1, same CSV format/precision as states) to a file. The no-run check happens before the usage check.

Arguments. path (required): rest-of-line, trimmed, may contain spaces.

On success. OK save: <absolute path> — the path echoed back is std::filesystem::absolute(path).

Errors. No prior run: 'ERR save: nothing to save (launch first)'. Empty path: 'ERR usage: save <path.csv>'. Open failure: 'ERR save: cannot open <path> for writing'.

Side effects. Writes/overwrites the CSV file.

The addpart grammar in full

Part types, their required and optional `key=value` parameters, and the placement-link vocabulary are tabulated in [Sections 2.1](#) and [2.2](#) — the tutorials in [Sections 3](#) and [5](#) exercise all of it against the real binary.

Part II

The Graphical Interface (Planned)

Part II is not yet written

The Qt6 GUI currently opens and saves `.qrd` designs, shows the part tree, imports and searches motors, and flies the same simulations — but rocket *authoring* (adding and editing parts) is CLI-only today. Part II will be written when the GUI's design editor lands. Until then, every workflow in this guide is available through `qtrocket-cli`, and `.qrd` files saved from the CLI open directly in the GUI and the visualizer.

A Reproducing this guide

Everything presented as output in this guide is regenerable from what ships in the repository, in the QtRocket documentation tradition of docs whose claims can be re-derived rather than trusted.

Transcripts. The five tutorial scripts live in `docs/qtrocket_userguide/scripts/` and run verbatim: `qtrocket-cli tutorial1.cli` (from a directory containing the data files each tutorial's setup box names). The typeset transcripts under `docs/qtrocket_userguide/transcripts/` were generated by running each script through the release binary and interleaving input with captured output — for every command, the script prefix up to that command was run in a fresh process and its output diffed against the previous prefix's, so each command is paired with exactly the lines it produced. Tutorial 3 exits with code 1 by design (it contains two intentional failures).

Screenshots. The four figures are `qtrocket-visualizer` displaying the `.qrd` files the tutorial sessions saved, captured at 1200×800 under a virtual X server (Xvfb, llvmpipe software GL). Tutorial 3's pair uses the Hi-Vis color scheme so the resolver's fixed error red is unmistakable against the orange nose and white body.

Plot data. The `data/*.dat` files are two-column extracts of the `saved` trajectory CSVs — the (t, z) columns, or (x, z) for the tilted flight — downsampled to roughly 150 rows (final row kept) and whitespace-separated for pgfplots, so the figures rebuild from committed data without rerunning simulations. The values themselves are unmodified.

This PDF. `make` in `docs/qtrocket_userguide/` runs `latexmk -pdf -shell-escape` (minted needs shell escape). The style is the QtRocket documentation series' shared identity — XCharter text, Inconsolata code, the `qr*` palette — inherited from the whitepapers under `docs/`.

Versions. Captured against QtRocket v0.2 (`development`, July 2026), motor data from the bundled `data/Aerotech.rse` and `tests/data/motors/estes_small.qmd`. Simulations are deterministic; rerunning the scripts reproduces every number shown, including the CSV contents.